ZipTok3D: High-Fidelity 3D Tokenization with Compact Token Prefixes

Mingda Lin* Weijie Wang* Zeyu Zhang Bowen Cui Yefei He Haoyu Zhao Yuanyu He
Donny Y. Chen Feng Chen† Bohan Zhuang†

Abstract.
Compact token sequences are essential for efficient 3D generation. However, existing 3D tokenizers typically organize latent representations either over spatial regions or as fixed-size sets of global tokens, both suffering sharp reconstruction degradation when compressed to extremely low token budgets. In this paper, we present **ZipTok3D**, a 3D tokenizer designed for high-fidelity reconstruction from extremely short token sequences. Its key idea is to organize object geometry into progressively informative global-token prefixes and unfold these compact representations through iterative decoding. Specifically, nested dropout randomly truncates the latent sequence after encoding during training and requires each retained prefix to reconstruct the complete object, thereby prioritizing essential geometric information in the leading tokens. The decoder then repeatedly applies a parameter-shared Transformer block to recover fine-grained geometry from each prefix without a separate generative sampling stage. With the same token dimension, ZipTok3D achieves reconstruction quality comparable to the 32-token COD-VAE baseline using only one token on ShapeNet and four on TRELLIS, yielding 32× and 8× shorter token sequences, respectively.

Project Page: https://forthloth.github.io/ziptok3d/
Correspondence: bohan.zhuang@zju.edu.cn
Code: https://github.com/forthloth/ziptok3d/tree/main/code_supplementary
Date: August 15, 2026
Keywords: 3D tokenization, 3D representation learning, neural fields, iterative refinement

* *Equal contribution.* † *Corresponding authors.*

Introduction

Recent 3D generation methods rely on tokenizers to convert complex object geometry into latent sequences, whose length substantially affects downstream generative modeling cost [1–4]. Existing 3D tokenizers either preserve spatial structure through locally anchored tokens [2, 4, 5] or compress object-wide geometry into a fixed set of global tokens [6–8]. While global representations produce shorter sequences, their reconstruction quality degrades sharply when the token budget is reduced to only a few tokens [6, 7]. This exposes a fundamental tension between latent sequence length and geometric fidelity: *can an entire 3D object be faithfully reconstructed from an extremely short token sequence?*

High-fidelity 3D reconstruction from only a few tokens poses a severe information bottleneck. With a moderate token budget, existing global 3D tokenizers can distribute complementary geometric information across multiple latent vectors. When the budget is reduced to only a few tokens, however, each token must summarize a substantially larger portion of the object, forcing global structure and fine-grained details to compete for severely limited latent capacity. Since these tokenizers optimize the complete latent set at a fixed budget [6, 7], they resolve this competition only implicitly through the final reconstruction objective, without explicitly determining which information should be preserved first. We therefore optimize a nested family of prefixes, requiring short prefixes to retain the object-wide geometry necessary for faithful reconstruction while allowing additional tokens to encode residual details.

Concentrating essential geometry into the leading tokens, however, addresses only the representation side of the problem. As the prefix becomes shorter, object-wide geometry is encoded in an increasingly compact form, placing a greater burden on the decoder to transform a few global vectors into a spatially detailed 3D representation. Conventional fixed-depth decoders perform this transformation through a single feed-forward process [6, 7] and may not fully exploit the information contained in extremely short prefixes. Faithful low-token reconstruction therefore requires not only prioritizing what is encoded in the leading tokens, but also progressively unfolding their compact geometric information during decoding.

To address these challenges, we present **ZipTok3D**, a 3D tokenizer that jointly learns progressively informative token prefixes and iteratively decodes the geometry they contain. During training, nested dropout [9] exposes the tokenizer to varying prefix lengths of the encoded latent sequence and requires each sampled prefix to reconstruct the complete input, encouraging the leading tokens to preserve object-wide geometry while subsequent tokens provide additional details. To effectively unfold the compact information encoded in short prefixes, the decoder recurrently applies a parameter-shared Transformer block [10, 11] under intermediate reconstruction supervision, progressively refining the spatial representation without introducing step-specific parameters. Unlike methods that rely on generative completion [12], ZipTok3D directly reconstructs the input geometry from each retained prefix without a separate sampling process. Using five refinement steps, ZipTok3D achieves reconstruction quality comparable to 32-token COD-VAE from only one token on ShapeNet and four tokens on TRELLIS, reducing representation length by 32× and 8×, respectively.

Our contributions are summarized as follows:

- We introduce **ZipTok3D**, a 3D tokenizer designed for faithful reconstruction from extremely short token sequences.
- We couple nested prefix tokenization with parameter-shared iterative refinement, enabling the leading tokens to preserve object-wide geometry and progressively unfolding their compact information into detailed 3D representations.
- Experiments show that one ZipTok3D token approaches COD-VAE-32 on ShapeNet, while four tokens surpass it on all reported TRELLIS metrics, using 32× and 8× fewer tokens.

Related Work

3D Latent Representations

Spatially organized methods associate latent features with points, grids, or hierarchical structures. LION uses hierarchical point-cloud features [13], 3DILG adopts irregular grids [5], and OctFusion uses octrees [14]. XCube builds sparse voxel hierarchies [15], while TRELLIS attaches features to occupied sparse-grid cells [16]. Recent methods further compress such representations through coarse voxel anchors, as in LATTICE [17], or sparse geometry-and-appearance latents, as in O-Voxel [18]. These representations preserve spatial support, but reducing it generally requires coarser or sparser structures.

Global methods instead compress object-wide geometry into compact token sets. VecSet obtains neural-field latents through cross-attention [6], while COD-VAE progressively compresses point features for triplane reconstruction [7]. SceneTok extends permutation-invariant token sets to multi-view scene modeling [19]. Other approaches use token hierarchies [8], apply multiscale residual quantization [1], or preserve surfaces via geometry-aware sampling [20].

Compact codes also support diverse decoders and generative models. Shape Tokens condition a flow-matching surface field [21], Kyvo uses quantized shape codes for multimodal autoregressive scene modeling [22], and FlashVDM accelerates VecSet-based generation through diffusion distillation and efficient implicit decoding [23]. However, fixed-budget objectives do not explicitly concentrate reconstructive information into short prefixes, so methods such as VecSet and COD-VAE can degrade sharply at very small token budgets.

Flexible-Length Tokenization

Flexible-length tokenizers learn representations that remain decodable at multiple sequence lengths. Nested dropout induces an information ordering by randomly truncating latent sequences during training [9]. Related image tokenizers learn prefix-decodable or causal one-dimensional representations [12, 24, 25], while adaptive methods allocate token budgets according to input complexity [26–29]. VideoFlexTok learns coarse-to-fine video prefixes with a generative flow decoder [30], and ReTok improves the utilization of later tokens in nested-dropout representations [31].

Recent 3D methods vary sequence length through spatial or semantic organization. OAT allocates octree tokens according to shape complexity [2], while SuperVoxelGPT adapts supervoxel size to local detail under a fixed generation order [4]. LoST orders tokens by semantic salience, with early prefixes specifying overall shape and later tokens adding instance details [3]. Its short prefixes condition diffusion-based completion rather than recover the encoded instance exactly. ZipTok3D instead learns nested reconstructive prefixes of global latents and decodes them directly through shared iterative refinement, without adaptive spatial partitioning or generative completion.

Iterative Refinement

A shared refinement module adds depth without introducing step-specific parameters. The Universal Transformer addresses the fixed depth of standard Transformers by recurrently applying shared self-attention and feed-forward layers [10]. ELT extends this idea to image and video generation, using weight-shared Transformer loops and intra-loop self-distillation to support different refinement depths [11].

Related strategies have been used in 3D to recover geometric detail from coarse or incomplete point clouds. The Cascaded Refinement Network recovers details missing from coarse predictions through cascaded coarse-to-fine refinement [32]. RFNet reduces the parameter and memory costs of dense completion by sharing operations across recurrent levels while progressively increasing point density and preserving observed details [33]. Together, these advances provide a useful basis for decoding detailed geometry from highly compressed token sequences.

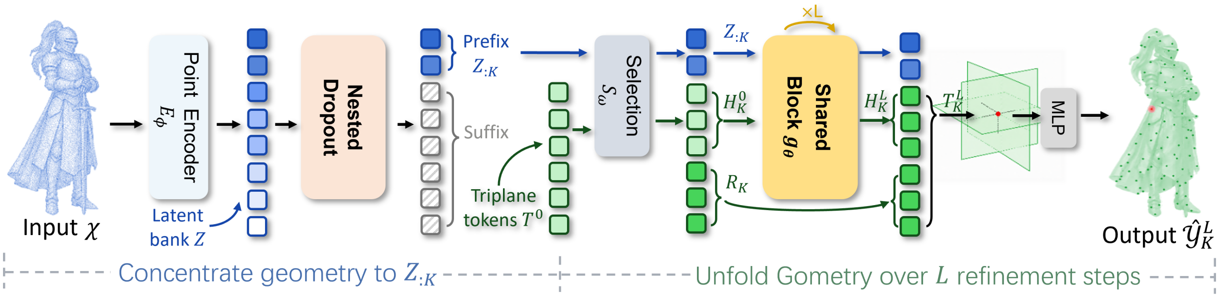

Figure 1 Overview of ZipTok3D. Nested dropout concentrates geometry into a reconstructive prefix $Z_{:K}$, while the parameter-shared block progressively unfolds it into triplane features over L refinement steps. Blue and green denote latent and triplane tokens, respectively; hatched tokens indicate the masked suffix.

Method

Overview

The central difficulty of low-token reconstruction is not compression alone. It combines two questions that a fixed-budget autoencoder does not separate: *what geometry remains available after the latent sequence is shortened*, and *how much computation is required to transform that compact representation into a spatial field*. We expose these factors as two inference controls. The retained prefix length K sets the latent sequence length, while the number of decoder iterations L controls the computation used to interpret that representation.

As shown in Figure 1, ZipTok3D changes how the latent sequence is organized and replaces single-pass triplane decoding with a reusable update rule. Let $\mathcal{X} = \{x_i\}_{i=1}^N \subset \mathbb{R}^3$ denote the N surface points sampled from an input shape. For an indexed collection of N_q query points $\mathcal{Q} = (q_1, \ldots, q_{N_q}) \in \mathbb{R}^{N_q \times 3}$, let $\mathcal{Y} = (y_1, \ldots, y_{N_q}) \in \{0,1\}^{N_q}$ denote the corresponding ground-truth occupancy labels, where y_j is the occupancy label of q_j. A progressive point encoder E_ϕ, parameterized by ϕ, maps $\mathcal{X}$ to a sequence of M latent vectors [7]:

$$Z = E_\phi(\mathcal{X}) = (z_1, \ldots, z_M) \in \mathbb{R}^{M \times d}, \qquad z_i \in \mathbb{R}^d, \tag{1}$$

where M is the maximum sequence length and d is the shared latent width. During training, nested dropout retains the prefix $Z_{:K} = (z_1, \ldots, z_K) \in \mathbb{R}^{K \times d}$, where $1 \leq K \leq M$.

For decoding, the learnable triplane tokens T^0 first interact with the retained prefix $Z_{:K}$ within the selection block $\mathcal{S}_\omega$ [7], which partitions the triplane tokens into the selected state H_K^0 and the redundant tokens R_K. This operation is denoted by $(H_K^0, R_K) = \mathcal{S}_\omega(T^0, Z_{:K})$. The retained prefix then conditions a parameter-shared Transformer block that refines H_K^0 for L iterations. After refinement, the updated state and the redundant tokens are restored to a dense triplane representation. The occupancy MLP queries this representation at the locations in $\mathcal{Q}$, producing the final prediction $\hat{\mathcal{Y}}_K^L$.

Learning a Reconstructive Prefix Code

A reconstruction objective applied only to the complete latent sequence constrains the information carried jointly by all tokens, but not how that information is allocated across token positions. The decoder may therefore rely on features distributed throughout the sequence, leaving its first few tokens insufficient for reconstructing the complete object. We instead optimize a nested family of prefixes across multiple token budgets. Each shorter prefix is contained in every longer one, requiring the leading tokens to support reconstruction at short lengths. Unlike arbitrary token dropout, this nested structure defines a consistent representation at each supported length and allows the sequence to be adjusted by truncation.

For each training example, the encoder first produces the complete latent sequence Z with maximum length $M = 128$. We uniformly sample the retained length from the exponentially spaced token budgets

$$\begin{aligned} \mathcal{K} &= \{1, 2, 4, 8, 16, 32, 64, 128\}, \\ K &\sim \mathrm{Unif}(\mathcal{K}). \end{aligned} \tag{2}$$

During training, the suffix $Z_{K+1:M}$ is masked from both triplane selection and iterative refinement, so the reconstruction objective must be realized using only $Z_{:K}$. At inference time, the masked suffix is removed, allowing the same checkpoint to operate at every trained token budget.

Iterative Global-to-Spatial Refinement

Prefix learning determines which evidence reaches the decoder, but it does not make the inverse mapping from a few global vectors to a detailed 3D field easy. With very small K, a single feed-forward pass must distribute object-level information across many spatial locations and recover the occupancy boundary in one transformation. Increasing the number of distinct decoder layers adds both computation and parameters while fixing the amount of computation after training.

We instead decode each retained prefix through L refinement steps. Starting from the compact triplane state H_K^0 produced by the selection module, the same parameter-shared Transformer block repeatedly updates the state while conditioning on the fixed prefix:

$$H_K^{\ell} = g_\theta\Big(H_K^{\ell-1}, Z_{:K}\Big), \qquad \ell = 1, \ldots, L. \tag{3}$$

The parameters θ and the retained prefix $Z_{:K}$ remain unchanged across iterations; only the triplane state is updated. Reusing the same block therefore increases the effective decoding depth without introducing step-specific parameters.

At refinement step ℓ, the updated state and the redundant tokens retained by the selection module can be restored to a complete triplane representation:

$$T_K^{\ell} = \mathcal{R}\Big(H_K^{\ell}, R_K\Big), \tag{4}$$

where $\mathcal{R}$ places the selected and redundant tokens back into their corresponding triplane locations. Given the query points in $\mathcal{Q}$ defined above, the occupancy predictions at refinement step ℓ are

$$\begin{aligned} \hat{y}_{K,j}^{\ell} &:= \sigma\Big(h_{\mathrm{occ}}(T_K^{\ell}, q_j)\Big), \qquad j = 1, \ldots, N_q, \\ \hat{\mathcal{Y}}_K^{\ell} &:= (\hat{y}_{K,1}^{\ell}, \ldots, \hat{y}_{K,N_q}^{\ell}) \in [0,1]^{N_q}. \end{aligned} \tag{5}$$

Here, h_{occ} is the shared occupancy MLP that predicts an occupancy logit from the triplane features sampled at q_j, and σ converts this logit into an occupancy probability. The final reconstruction is $\hat{\mathcal{Y}}_K^L$, while $\hat{\mathcal{Y}}_K^{\ell}$ for $\ell < L$ provides an intermediate prediction along the same refinement trajectory. This recurrent process progressively unfolds the geometric information concentrated in the retained prefix without introducing an additional latent sampling stage.

Training Objective

Weight sharing alone does not make every refinement state a valid reconstruction. If only the final iteration were supervised, earlier states could remain unconstrained as reconstruction outputs. We therefore supervise the final output together with one randomly sampled intermediate output from the same refinement trajectory.

For a sampled prefix length K, the same retained prefix is used throughout all L refinement steps. We partition the query points in $\mathcal{Q}$ into two disjoint subsets: the uniformly sampled volume queries $\mathcal{Q}_{\mathrm{vol}}$ and the near-surface queries $\mathcal{Q}_{\mathrm{near}}$. Let $\mathcal{I}_{\mathrm{vol}}$ and $\mathcal{I}_{\mathrm{near}}$ denote their corresponding index sets, with cardinalities N_{vol} and N_{near}, respectively, so that $N_q = N_{\text{vol}} + N_{\text{near}}$. The reconstruction loss at refinement step ℓ is

$$\begin{aligned}\mathcal{L}_{\text{rec}}(\hat{\mathcal{Y}}_K^\ell, \mathcal{Y}) = \frac{1}{N_{\text{vol}}} \sum_{j \in \mathcal{I}_{\text{vol}}} \text{BCE}(\hat{y}_{K,j}^\ell, y_j) \\ + \frac{\lambda_{\text{near}}}{N_{\text{near}}} \sum_{j \in \mathcal{I}_{\text{near}}} \text{BCE}(\hat{y}_{K,j}^\ell, y_j).\end{aligned} \tag{6}$$

Following COD-VAE [7], we set $\lambda_{\text{near}} = 0.1$.

To stabilize the joint optimization of triplane initialization and token selection, we follow COD-VAE [7] and retain its two auxiliary losses:

$$\mathcal{L}_{\text{aux}} = \lambda_{\text{init}} \mathcal{L}_{\text{init}} + \lambda_{\text{unc}} \mathcal{L}_{\text{unc}}.$$

We use $\lambda_{\text{init}} = 0.5$ and $\lambda_{\text{unc}} = 0.001$. Here, $\mathcal{L}_{\text{init}}$ directly supervises the initial triplane prediction, while $\mathcal{L}_{\text{unc}}$ trains the uncertainty estimates used for token selection.

To provide effective supervision for the intermediate states, we uniformly sample a refinement step $\ell \in \{1, \ldots, L-1\}$, where $L \geq 2$. Its prediction $\hat{\mathcal{Y}}_K^\ell$ receives direct reconstruction supervision and intra-loop self-distillation [11]. Using the same query weighting as the reconstruction loss, the distillation loss aligns the intermediate and detached final occupancy predictions:

$$\begin{aligned}\mathcal{L}_{\text{dist}}(\hat{\mathcal{Y}}_K^\ell, \text{sg}(\hat{\mathcal{Y}}_K^L)) = \frac{1}{N_{\text{vol}}} \sum_{j \in \mathcal{I}_{\text{vol}}} \text{BCE}(\hat{y}_{K,j}^\ell, \text{sg}(\hat{y}_{K,j}^L)) \\ + \frac{\lambda_{\text{near}}}{N_{\text{near}}} \sum_{j \in \mathcal{I}_{\text{near}}} \text{BCE}(\hat{y}_{K,j}^\ell, \text{sg}(\hat{y}_{K,j}^L)).\end{aligned} \tag{7}$$

Here, $\text{sg}(\cdot)$ denotes stop gradient. The detached final probability serves as a soft occupancy target for the intermediate prediction.

For the sampled token budget K and intermediate depth ℓ, the overall training objective is

$$\begin{aligned}\mathcal{L}_{K,\ell} = \underbrace{\mathcal{L}_{\text{rec}}(\hat{\mathcal{Y}}_K^L, \mathcal{Y}) + \mathcal{L}_{\text{aux}}}_{\text{final and auxiliary supervision}} \\ + \beta \underbrace{\left[\alpha_t \, \mathcal{L}_{\text{rec}}(\hat{\mathcal{Y}}_K^\ell, \mathcal{Y}) + (1-\alpha_t) \, \mathcal{L}_{\text{dist}}(\hat{\mathcal{Y}}_K^\ell, \text{sg}(\hat{\mathcal{Y}}_K^L))\right]}_{\text{intermediate supervision}}.\end{aligned} \tag{8}$$

Here, $\beta = 0.5$ controls the contribution of intermediate supervision. At optimizer step t, we set $\alpha_t = \max(0, 1 - t/T_\alpha)$, where t counts completed optimizer updates and T_α is the total number of scheduled updates after accounting for distributed training and gradient accumulation.

Experiments

Experimental Setup

Datasets

We evaluate 3D reconstruction on ShapeNet-v2 (ShapeNet) [34] and TRELLIS-500K (TRELLIS) [16]. For ShapeNet, we follow the preprocessing and split of COD-VAE [7]; the evaluation split contains 1,283 objects from 55 categories. For TRELLIS, we retain assets with polygonal meshes, apply the watertight preprocessing of VecSet [6], and partition the data with seed 42, yielding an evaluation split of 2,613 assets. Supplementary Sec. S2 reports the source composition

Method	Tok.	ShapeNet			TRELLIS		
		IoU↑	CD↓	F1↑	IoU↑	CD↓	F1↑
3DILG	512	95.9	<u>0.013</u>	**98.0**	–	–	–
VecSet	32	87.8	0.021	91.3	–	–	–
VecSet	64	91.2	0.017	94.7	–	–	–
VecSet	512	96.3	<u>0.013</u>	**98.0**	71.47	0.0249	88.81
COD-VAE	2	77.7	0.032	80.9	45.85	0.0674	53.95
COD-VAE	32	<u>97.1</u>	**0.012**	97.8	75.25	0.0172	95.67
COD-VAE	64	**97.5**	**0.012**	**98.0**	**75.75**	0.0168	**95.98**
ZipTok3D	1	96.8	**0.012**	97.8	75.18	0.0168	95.81
ZipTok3D	2	96.9	**0.012**	97.8	75.22	<u>0.0167</u>	95.86
ZipTok3D	4	96.9	**0.012**	<u>97.9</u>	<u>75.31</u>	**0.0166**	<u>95.92</u>

Table 1 Reconstruction results on ShapeNet and TRELLIS under the common evaluation protocol described in the experimental setup. Tok. denotes latent sequence length, and IoU and F1 are percentages. All ZipTok3D operating points for a dataset are evaluated from the same checkpoint using five refinement steps. Best results within each dataset and metric are shown in bold, and second-best results are underlined; ties at the displayed precision share the same formatting. A dash denotes an unavailable result when the corresponding public checkpoint was not released.

and exact asset manifest for this split.

Baselines

For quantitative comparison, we consider 3DILG [5], VecSet [6], and COD-VAE [7], which support deterministic reconstruction under fixed, shape-independent token budgets. All ShapeNet reconstruction results follow the same test split and evaluation pipeline established by COD-VAE, with additional operating points evaluated using the official implementations. All TRELLIS results use the same split.

Models and Training

The tokenizer takes 2,048 surface points and produces at most 128 latent tokens of width 512. At inference, ZipTok3D receives the actual $K \times 512$ prefix without suffix padding; COD-VAE uses the same token width. The decoder reuses one six-layer Transformer block across refinement passes, and the main results use five passes to produce a 128×128 triplane. Supplementary Secs. S3 provide detailed architecture specifications, recurrent refinement inputs, and definitions of the inherited auxiliary losses.

We train the tokenizer in FP16 with AdamW using a learning rate of 10^{-4}, weight decay 0.01, and an effective batch size of 672 on four A800 GPUs. ShapeNet and TRELLIS training run for 1,000 and 300 epochs, respectively. We select checkpoints by the highest validation query IoU. Each tokenizer and ablation configuration is trained once using seed 123456. Supplementary Sec. S2 specifies the optimizer parameters, per-GPU batch size, gradient accumulation, learning-rate schedules, loss weights, and model-selection protocol.

For class-conditioned generation, a causal second-stage VAE maps each $K \times 512$ tokenizer prefix to a $K \times 32$ latent sequence without changing its length. At the reported operating point, the EDM models the exact 2×32 sequence without suffix padding, whereas COD-VAE-32 uses a 32×32 stage-2 sequence. Supplementary provide the stage-2 training protocol and detailed architectural specifications of the prefix VAE and EDM, respectively.

Evaluation

Following COD-VAE, reconstruction is evaluated using query IoU over 500,000 volume queries and mesh CD and F1 computed from 128^3 marching-cubes reconstructions. For class-conditioned generation, we evaluate airplane, car, chair, table, and rifle using 2,000 generated shapes per category and report MMD-CD, COV-CD, and 1-NNA-CD. The diffusion models use 18 sampling steps, whereas 3DILG uses its native 512-step autoregressive sampler.

Efficiency is measured using trained checkpoints on one NVIDIA H20 GPU with batch size 16 and FP32 inference, with TF32 and Flash SDP disabled. ZipTok3D always receives an exact prefix without suffix padding. Sampling throughput covers latent generation, while full throughput additionally includes tokenizer decoding and the complete 128^3 occupancy-field query. Supplementary Secs. S4 provide detailed evaluation settings, reference-set construction, sampling parameters, query chunking, CUDA timing, and peak-memory measurement.

Sampling throughput covers latent generation only, whereas full throughput additionally includes decoding and the complete 128^3 occupancy-field query in chunks of 131,072 points. CUDA events are used for timing. Decoder latency discards three warmup batches and averages ten measured batches; generation efficiency discards five warmup batches and averages thirty measured batches. Peak memory is the maximum allocated, rather than reserved, GPU memory during measured full inference. Timing excludes model initialization, data loading, CPU–GPU transfers, marching cubes, and file writing.

Main Results

Reconstruction from Extremely Short Prefixes

Table 1 shows that ZipTok3D retains high-fidelity geometry at sequence lengths where fixed-budget tokenizers deteriorate. On ShapeNet, the one-token setting attains the same rounded CD and F1 values as COD-VAE-32 and remains within 0.3 IoU points, while using a 32× shorter latent sequence. By contrast, COD-VAE degrades sharply at two tokens, showing that the result does not follow from simply reducing the size of a fixed latent set. On the more diverse TRELLIS split, the four-token setting yields a similar query-IoU point estimate and improves CD and F1 over COD-VAE-32 with an 8× shorter latent sequence.

For this primary TRELLIS comparison, we use a paired, object-level nonparametric percentile bootstrap with 20,000 resamples and seed 123456. Relative to COD-VAE-32, ZipTok3D with $K = 4$ changes query IoU by $+0.06$ points (95% CI $[-0.08, +0.20]$), reduces mesh CD by 0.00058 (95% CI $[0.00039, 0.00080]$), and increases mesh F1 by 0.25 points (95% CI $[0.16, 0.34]$). The query-IoU interval includes zero, whereas the CD and F1 intervals exclude zero and favor ZipTok3D. Supplementary Sec. S4 details the resampling procedure and clarifies that these intervals reflect variation across evaluation objects rather than independent training runs.

Figure 2 further shows that one- and two-token prefixes preserve global part layouts and thin structures, including bench legs, flower stems, and bridge towers, across both datasets.

Class-Conditioned Generation

We evaluate class-conditioned generation from an exact two-token 2×32 latent code, with each generated code decoded using five shared refinement passes.

Compared with COD-VAE-32, ZipTok3D-2 uses a 16× shorter stage-2 latent sequence while remaining close on all three distribution metrics. Under the common efficiency protocol, it provides higher sampling throughput with similar full throughput and peak memory, and is substantially faster than the VecSet baselines across the full pipeline. These results show that the compact prefix remains effective for downstream class-conditioned generation.

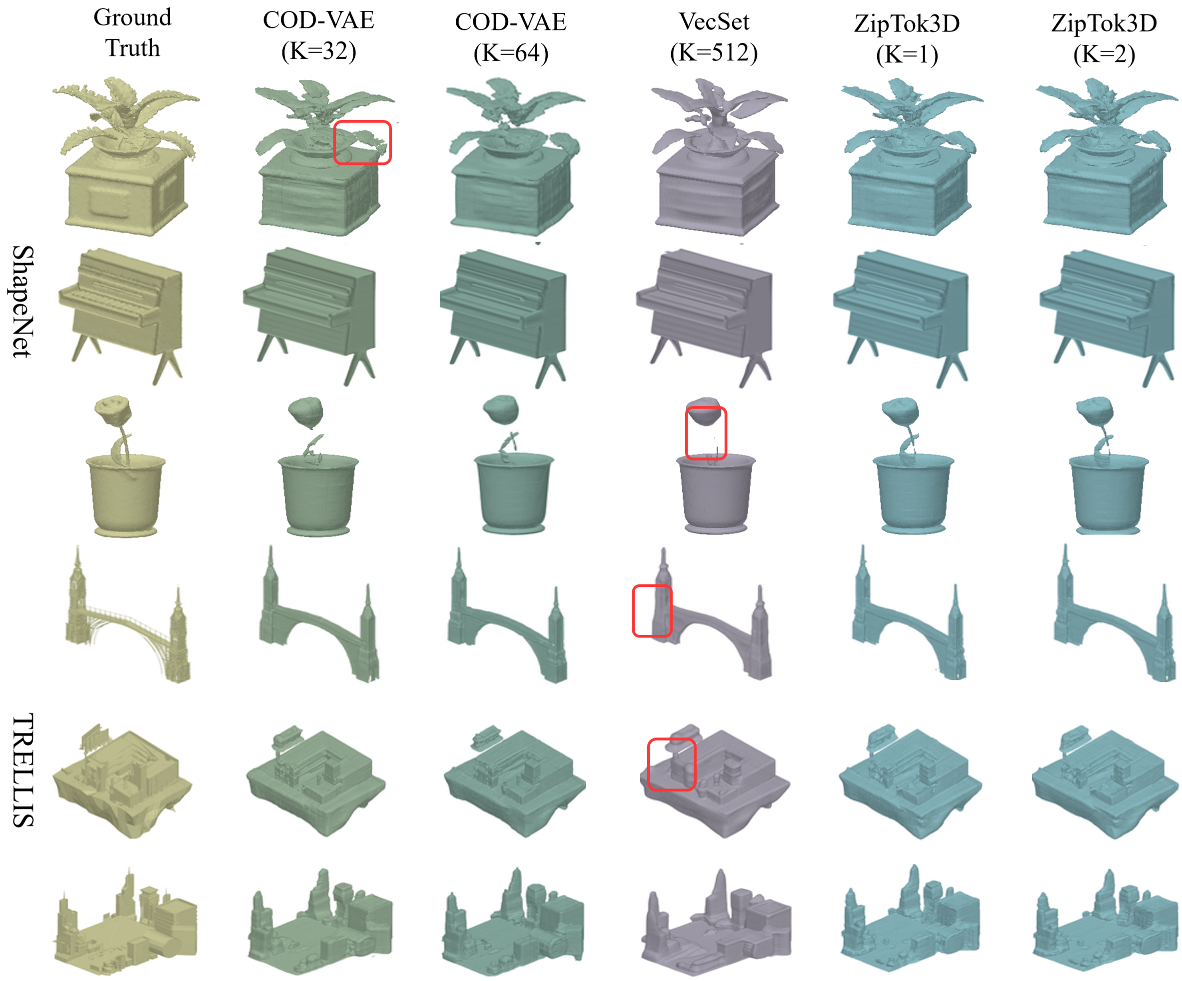

Figure 2 Qualitative reconstructions on ShapeNet (top) and TRELLIS (bottom). Columns show ground truth, COD-VAE with $K = 32$ and $K = 64$, VecSet with $K = 512$, and ZipTok3D with $K = 1$ and $K = 2$. Both ZipTok3D settings use five refinement steps.

	Distribution			Efficiency		
Method	MMD↓	COV↑	1-NNA↓	Samp.↑	Full↑	Mem.↓
3DILG	6.270	58.16	61.49	0.24	0.22	26.18
VecSet-32	<u>4.999</u>	84.51	54.32	28.39	6.59	25.68
VecSet-64	5.090	84.89	55.04	25.60	6.11	26.19
VecSet-512	**4.807**	**85.20**	<u>53.78</u>	4.27	2.14	33.24
COD-VAE-32	5.020	<u>84.96</u>	**53.11**	<u>46.02</u>	<u>36.14</u>	<u>2.26</u>
ZipTok3D-2	5.142	84.91	54.21	**50.62**	**36.64**	**2.20**

Table 2 Class-conditioned generation on five ShapeNet categories. All distribution metrics are CD-based. MMD is scaled by 10^{-3}, while COV and 1-NNA are percentages, with 50% being ideal for 1-NNA. Samp. measures latent generation alone, whereas Full additionally includes tokenizer decoding and the complete 128^3 occupancy-field query; both are reported in samples per second. Mem. is peak allocated memory in GiB during the full pipeline. Suffixes indicate stage-2 token count. Bold and underlined values denote the best and second-best results, respectively.

Ablation Study

Component Ablation

We examine prefix training, shared iterative refinement, and intermediate supervision through matched local controls on ShapeNet at $K = 2$. *Prefix only* retains the original 12-layer, single-pass COD-VAE decoder and differs from COD-VAE only by applying nested dropout during training. The remaining variants use the same six-layer decoder as ZipTok3D. *w/o iterative refinement* applies the block once, *w/o inter. sup.* applies it for five passes with supervision only on the final output, and the full model additionally supervises an intermediate prediction.

			Reconstruction			Decoder Cost	
Variant	K	Decoder	IoU↑	CD↓	F1↑	Params↓	Lat.↓
COD-VAE	2	12×1	77.7	0.032	80.9	<u>39.3</u>	<u>1.47</u>
Prefix only	2	12×1	92.3	0.015	95.8	<u>39.3</u>	<u>1.47</u>
w/o iter. ref.	2	6×1	91.9	0.017	94.2	**23.5**	**0.98**
w/o inter. sup.	2	6×5	<u>96.6</u>	<u>0.013</u>	<u>97.6</u>	**23.5**	2.65
Full ZipTok3D	2	6×5	**96.9**	**0.012**	**97.8**	**23.5**	2.65

Table 3 Ablation study on ShapeNet at $K = 2$. *w/o iter. ref.* denotes the variant without iterative refinement, in which the six-layer block is applied once. *w/o inter. sup.* retains five refinement passes but supervises only the final output. In the decoder column, $d \times p$ denotes d distinct Transformer layers applied for p passes; shared layers are counted once. Params and Lat. denote the number of trainable decoder parameters in millions and decoder latency in milliseconds per shape, respectively. IoU and F1 are percentages. Best results are shown in bold, and second-best results are underlined; ties share the same formatting.

At the matched 12×1 architecture and decoder cost, *Prefix only* improves all three reconstruction metrics over COD-VAE, supporting the effect of nested prefix training in the original decoder. Within the six-layer architecture, comparison between *w/o iterative refinement* and *w/o inter. sup.* shows that repeated application of the shared block provides the main additional improvement without adding decoder parameters, although it increases latency. Intermediate supervision provides a modest further improvement in final reconstruction without changing inference cost. These results identify prefix organization and shared refinement as the primary contributors, with intermediate supervision providing a complementary benefit.

Effect of Prefix Length and Refinement Depth

We evaluate all supported combinations of prefix length and refinement depth using one checkpoint per dataset. Figure 3 shows the query-IoU results at representative refinement depths.

The two factors play complementary roles. Longer prefixes provide the largest benefit under a single decoding pass, whereas performance becomes progressively less sensitive to K after three to five passes. Conversely, additional refinement produces the largest gains at $K = 1$ and $K = 4$, while longer prefixes already provide strong single-pass reconstructions. TRELLIS benefits from deeper refinement than ShapeNet, but both datasets approach saturation by $L = 5$. Thus, additional tokens primarily compensate for shallow decoding, whereas recurrent computation is most valuable in the extremely low-token regime. The corresponding mesh CD and F1 curves exhibit the same interaction pattern under the same settings (Supplementary Sec. S1).

Figure 4 illustrates this progression. With $L = 1$, the decoder captures portions of the global structure but may omit components or produce fragmented surfaces. By $L = 3$, it recovers the major structure and produces a coherent reconstruction. The fifth pass primarily sharpens thin structures, boundaries, and local details. This coarse-to-fine progression is consistent with the diminishing quantitative gains near $L = 5$.

Figure 3 Effect of prefix length K and refinement depth L on query IoU for ShapeNet (top) and TRELLIS (bottom). The plots vary K at fixed L (left) and L at fixed K (right).

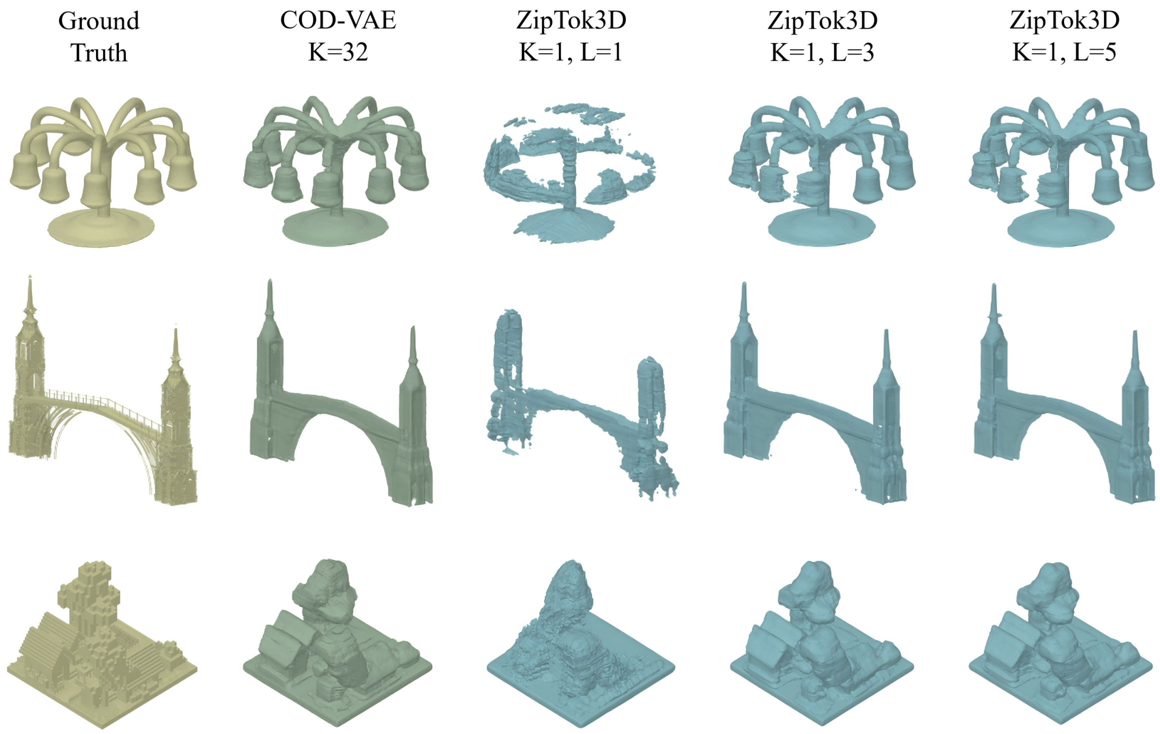

Figure 4 Qualitative refinement at $K = 1$. Columns show ground truth, COD-VAE with $K = 32$, and ZipTok3D reconstructions with $K = 1$ after $L = 1$, $L = 3$, and $L = 5$ refinement passes.

These results suggest that the leading tokens retain the evidence needed to recover global structure, but realizing it as complete geometry requires repeated spatial refinement. Longer prefixes are therefore most beneficial under shallow decoding, whereas deeper refinement more fully unfolds the information concentrated in short prefixes.

Conclusion

We presented ZipTok3D, a 3D tokenizer designed for high-fidelity reconstruction from extremely short latent sequences. Nested prefix training encourages the leading tokens to preserve object-wide geometry, while parameter-shared iterative refinement progressively transforms this compact information into a detailed spatial representation without introducing step-specific parameters. Experiments show that one ZipTok3D token approaches the reconstruction quality of 32-token COD-VAE on ShapeNet, while four tokens yield similar performance on TRELLIS, reducing token counts by 32× and 8×, respectively. The ablation results confirm that prefix organization and iterative refinement provide complementary benefits in the few-token regime. Our two-token stage-2 representation also supports class-conditioned generation with performance close to COD-VAE-32, indicating that the compact prefixes remain useful for downstream generative modeling. Future work will investigate generative models that better exploit short latent sequences and adaptively allocate prefix length and refinement depth according to each input.

S1 Prefix-Length and Refinement-Depth Analysis

S1.1 Complete Metric Sweeps

To characterize how prefix length and refinement depth jointly affect reconstruction, we evaluate all supported prefix lengths $K \in \{1, 2, 4, 8, 16, 32, 64, 128\}$ and refinement depths $L \in \{1, 2, 3, 4, 5, 6\}$. This extends the query-IoU analysis in the main paper to the complete operating range with two surface metrics. All operating points within each dataset use the same selected checkpoint, and the decoder receives the exact $K \times 512$ prefix without suffix padding.

Figures A and B provide complementary axis-wise views of this interaction for mesh F1@0.02 and mesh CD. When refinement is shallow, increasing K yields the largest gains. Conversely, when the prefix is short, increasing L is most beneficial. This asymmetric interaction is observed on both datasets and is consistent with the query-IoU results in the main paper, showing that additional prefix tokens and refinement passes contribute most strongly in different regimes.

Figure C reports the complete joint grids for query IoU, mesh F1@0.02, and mesh CD. Across the evaluated range, the full sweeps confirm that reconstruction is most sensitive to prefix length under shallow decoding, whereas the benefit of additional refinement is concentrated at small K. At the operating points used in the main paper, the differences between $L = 5$ and $L = 6$ are small across all three metrics, supporting the use of five refinement passes for evaluation.

S1.2 Per-Object Refinement Behavior

Dataset-level mean gains do not show whether refinement benefits most objects or only a small subset. Table A therefore reports the fraction of objects whose IoU, CD, or F1 improves over an early refinement transition, $L = 1 \to 3$, and a later transition, $L = 3 \to 5$, at two short-prefix settings, $K \in \{1, 4\}$.

			Objects improved (%)		
Dataset	K	Transition	IoU↑	CD↓	F1↑
ShapeNet	1	$1 \to 3$	100.0	99.3	97.6
	1	$3 \to 5$	88.2	81.1	58.1
	4	$1 \to 3$	100.0	98.7	94.8
	4	$3 \to 5$	89.5	81.5	55.7
TRELLIS	1	$1 \to 3$	98.6	99.8	99.8
	1	$3 \to 5$	91.2	91.8	84.7
	4	$1 \to 3$	98.5	99.7	99.4
	4	$3 \to 5$	90.5	90.7	83.4

Table A Per-object improvement rates across selected refinement intervals. An object is counted as improved if its IoU or F1 strictly increases or its CD strictly decreases. Ties and regressions remain in the denominator but are not counted as improvements. Values are rounded to one decimal place.

Across both datasets, increasing L from 1 to 3 improves a large majority of objects on every metric, indicating that the aggregate gains are not driven by a small subset. Increasing L further from 3 to 5 still improves more than half of the objects in every setting, but the improvement rates are lower and more metric-dependent, with the weakest consistency for ShapeNet mesh F1. Early refinement therefore provides broadly shared benefits, whereas later refinement remains useful for a smaller and more metric-dependent fraction of objects.

S2 Experimental Setup Details

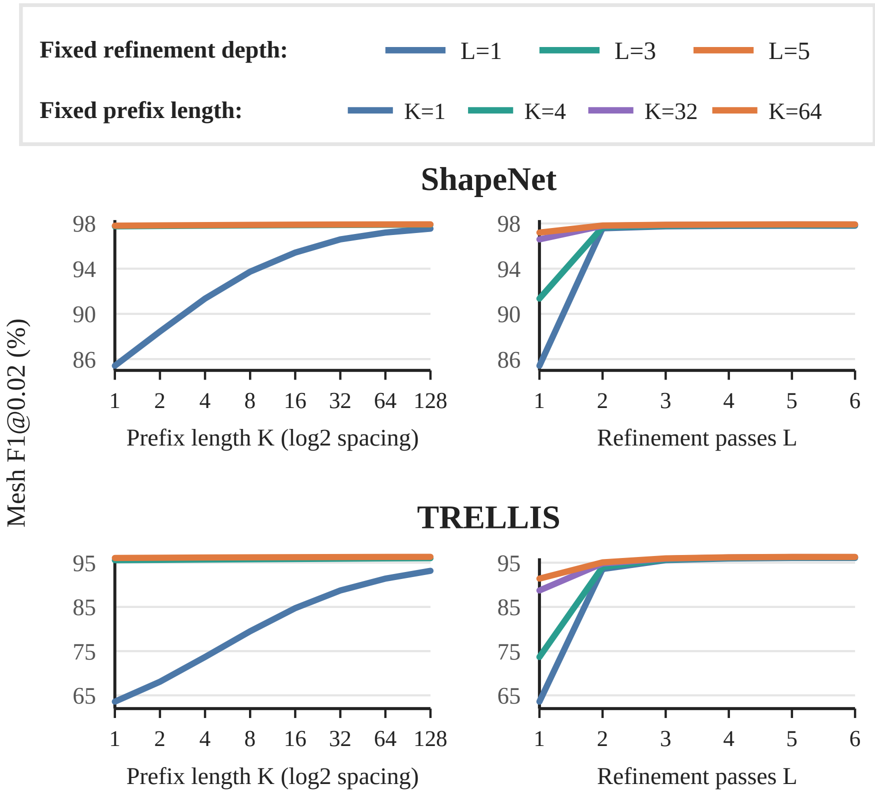

Figure A Mesh F1@0.02 across prefix lengths K and refinement depths L on ShapeNet (top) and TRELLIS (bottom).

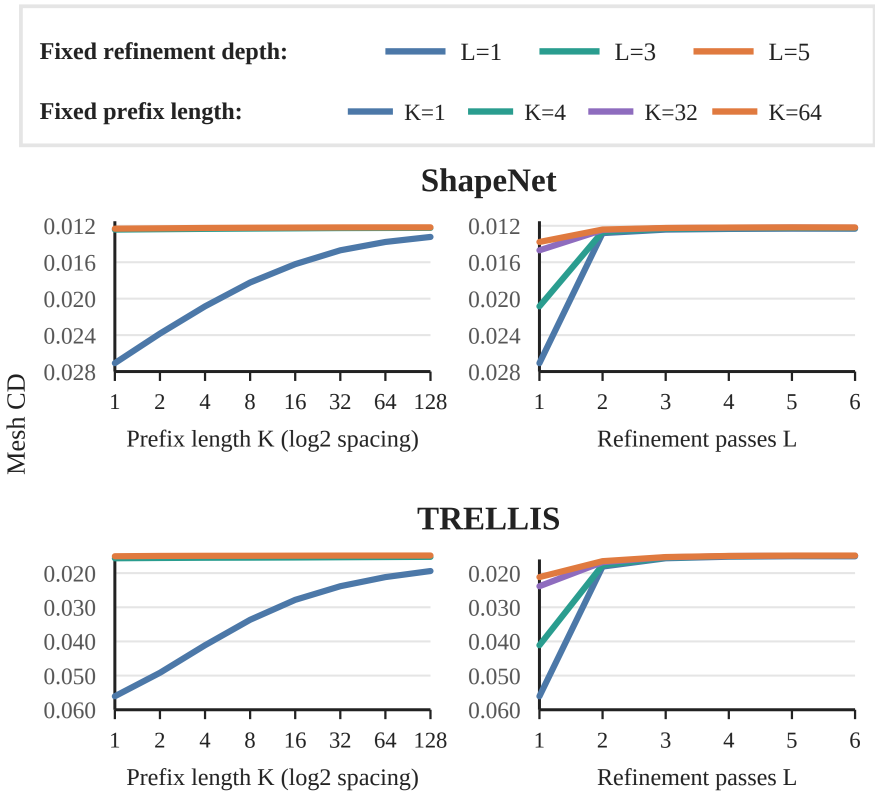

Figure B Mesh CD across prefix lengths K and refinement depths L on ShapeNet (top) and TRELLIS (bottom). Lower is better.

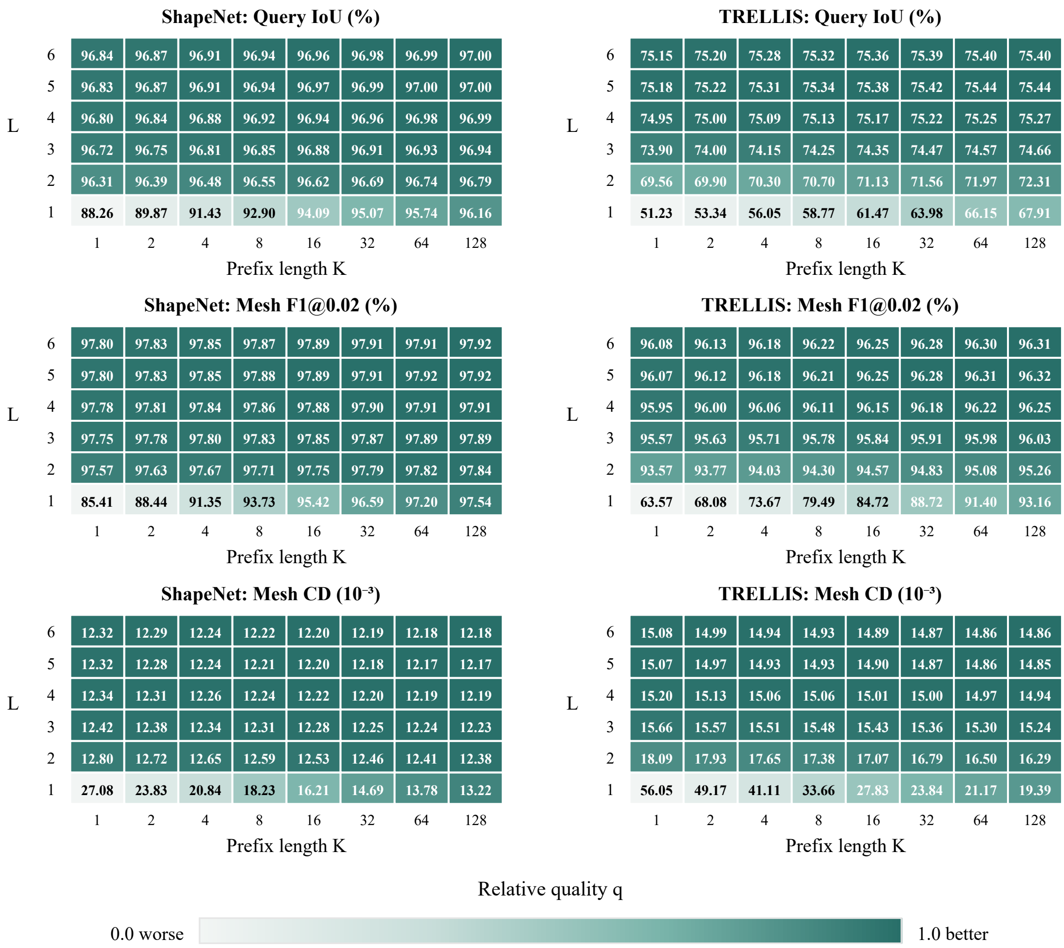

Figure C Complete ShapeNet (left) and TRELLIS (right) sweeps for refinement depths $L = 1, \ldots, 6$ and all supported prefix lengths. Rows report query IoU, mesh F1@0.02, and mesh CD. Cell values are dataset means; CD is reported in units of 10^{-3}. To orient all metrics so that better values receive darker colors, we define the residual from the optimum as $r = 100 - m$ for percentage-valued IoU and F1 and as $r = m$ for CD. Within each dataset–metric panel, we apply standard min–max normalization to $-\log r$. If $r_{\min}$ and $r_{\max}$ are the minimum and maximum residuals over its displayed (K, L) cells, the resulting color score is $q = \log(r_{\max}/r)/\log(r_{\max}/r_{\min})$. Thus, $q = 1$ denotes the best cell and $q = 0$ the worst. Darker colors indicate larger q.

S2.1 Datasets, Preprocessing, and Splits

ShapeNet. We use ShapeNetCore-v2 [34] with the 55-category preprocessing and split manifests released by 3DShape2VecSet [6] and adopted by COD-VAE [7]. The Stage-1 tokenizer, prefix VAE, and class-conditional EDM share the same partition: 48,597 objects are used for training, 2,592 for validation and model selection, and a disjoint 1,283-object test set for final evaluation. Reconstruction is evaluated on the complete test set, whereas generation metrics use its category-specific subsets for airplane, car, chair, table, and rifle. These role-based names correspond to the subsets stored as *test* and *validation*, respectively, in the released manifests. For each object, the released data provide separate pools of uniform volume queries and near-surface queries, their occupancy labels, a normalization scale, and a dense reference surface point cloud. Following COD-VAE, we consolidate the records by split while preserving category and object identifiers. All manifest entries used in the reported experiments are successfully converted. Query coordinates are stored in FP16 for training and FP32 for both held-out splits, while surface points remain in FP32. During training, independent axis scales are sampled uniformly from $[0.75, 1.25]$, the shape is renormalized to the unit cube, and Gaussian noise with standard deviation 0.005 is added to the surface points before clipping to $[-1, 1]$. The same geometric scaling is applied to the occupancy queries. Neither held-out split is augmented.

TRELLIS. TRELLIS-500K [16] provides metadata for 500,777 training assets: 168,307 Objaverse-XL Sketchfab assets, 311,843 Objaverse-XL GitHub assets, 4,485 ABO assets, 9,472 3D-FUTURE assets, and 6,670 HSSD assets. Its separate 3,229-object Toys4K evaluation set is not used. Our preprocessing starts from the available downloaded assets rather than the complete metadata catalogue. We merge supported polygonal components and apply the point-cloud-utils watertight conversion used by the VecSet preprocessing pipeline at resolution 50,000 with seed 0. The watertight mesh is centered at its bounding-box center and scaled so that its maximum vertex radius is one. Each successful object is represented by 100,000 surface points, 500,000 volume queries sampled uniformly from the radius-$\sqrt{3}$ sphere, and 500,000 near-surface queries divided equally between Gaussian perturbation scales 0.005 and 0.05. Occupancy is one where the signed distance is nonpositive and zero otherwise. Per-object sampling is deterministic from seed 42 and the object identifier. Inputs above 50,000 faces are decimated to that target before watertight conversion when decimation succeeds. The preprocessing additionally sets a 900-second timeout and a 32-GiB memory limit per worker. An asset is excluded if it lacks a supported, nonempty polygonal mesh, contains nonfinite geometry, or fails mesh conversion, normalization, surface sampling, signed-distance evaluation, or the stated resource limits.

With seed 42, the successfully preprocessed assets are partitioned into 97% training, 2% validation, and 1% test subsets. The resulting test split contains 2,613 objects: 57 ABO, 856 Objaverse-XL GitHub, and 1,700 Objaverse-XL Sketchfab assets. The anonymous code release provides the source category and exact object identifier of every asset in this test split, which is used for all reconstruction, bootstrap, and adaptive-budget analyses.

S2.2 Baseline Sources and Operating Points

We compare with 3DILG [5], VecSet [6], and COD-VAE [7]. These methods support deterministic reconstruction at fixed, shape-independent token budgets. All ShapeNet comparisons use the common 1,283-object test split and the COD-VAE metric implementation. The ShapeNet COD-VAE-32/64 checkpoints are inherited from the official COD-VAE release, whereas COD-VAE-2 is trained using the released implementation. On TRELLIS, COD-VAE-2/32/64 are initialized from their corresponding ShapeNet checkpoints and fine-tuned on the common preprocessed training pool. These checkpoints are selected by the highest mean query IoU on the same validation split used by ZipTok3D. VecSet-512 uses the inherited ShapeNet model without TRELLIS-specific training. TRELLIS results are not reported for 3DILG or the shorter VecSet operating points. All reported TRELLIS methods use the common test split and evaluation protocol described above.

The reconstruction comparisons use 512 tokens for 3DILG; 32, 64, and 512 tokens for VecSet; and 2, 32, and 64 tokens for COD-VAE. ZipTok3D is evaluated with exact prefixes of 1, 2, and 4 tokens. The class-conditioned generation comparison uses the native stage-2 latent lengths of the baselines and the exact two-token stage-2 representation of ZipTok3D.

S2.3 Stage-1 Optimization and Model Selection

For both datasets, each training example samples 2,048 points from the stored surface pool as encoder input and separately samples 4,096 volume queries and 4,096 near-surface queries for reconstruction supervision. We optimize the Stage-1 tokenizer with AdamW using $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, weight decay 0.01, and an initial learning rate of 10^{-4}. Training uses FP16 mixed precision and clips the global gradient norm at 0.5. Each of four A800 GPUs processes 56 examples, and gradients are accumulated over three steps, giving an effective batch size of 672.

ShapeNet training runs for 1,000 epochs with a 50-epoch linear warmup followed by cosine learning-rate decay. TRELLIS training runs for 300 epochs at a constant learning rate. Every configuration is trained once with seed 123456. For ShapeNet, checkpoints are selected by the highest mean query IoU on the 2,592-object validation split. For TRELLIS, the same criterion is evaluated on the validation split. The implementation uses Python 3.9, PyTorch 2.1.0, and CUDA 12.2. Training-time refinement and supervision are specified in Section S3.4.

S2.4 Prefix VAE Optimization and Model Selection

The prefix VAE uses AdamW with learning rate 10^{-4}, weight decay 0.01, global batch size 376, FP16 mixed precision, and gradient clipping at 0.5. It is trained for 100 epochs, with the learning rate halved at epochs 60, 70, 80, and 90. It is trained on the same 48,597-object ShapeNet training split as the Stage-1 tokenizer. Training uses four A800 GPUs and seed 123456. We select the checkpoint with the highest query IoU on the shared 2,592-object ShapeNet validation split.

S2.5 EDM Optimization and Model Selection

The EDM uses AdamW with learning rate 10^{-4}, weight decay 0.05, global batch size 256, FP16 mixed precision, and gradient clipping at 1.0. Its 1,000-epoch schedule uses a 40-epoch linear warmup followed by cosine decay to 10^{-6}. Training uses four A800 GPUs and seed 123456. We set $p_{\text{mean}} = -1.2$, $p_{\text{std}} = 1.2$, and $\sigma_{\text{data}} = 1$. The EDM is trained on latent arrays cached from the shared ShapeNet training split, and its checkpoint is selected by the lowest denoising loss on the shared validation split.

S3 Architecture and Objective Specifications

S3.1 Stage-1 Tokenizer Architecture

Unless noted otherwise, the Stage-1 components follow the released COD-VAE design [7]. ZipTok3D introduces nested prefix training, the shared refinement block, and intermediate supervision.

The point encoder receives 2,048 surface points and forms 512 patches by farthest-point sampling. It has four progressive blocks, each containing three self-attention layers. The encoder width is 512, with eight attention heads, a GEGLU MLP ratio of 4, and dropout 0.1. It produces a latent bank with maximum length $M = 128$ and width 512. The initial latent queries are the point embeddings of 128 farthest-point-sampled surface positions, without a separate learned latent-position table.

For each training example, one prefix length is sampled uniformly from $\{1, 2, 4, 8, 16, 32, 64, 128\}$. All suffix positions are masked from every decoder attention operation. The decoder starts from 768 learned triplane tokens T^0, corresponding to three 32-channel planes at spatial resolution 128×128 with 8×8 patches. To predict occupancy, features are bilinearly sampled from the three planes, summed, and processed by a $32 \to 32 \to 1$ MLP with GELU.

S3.2 Triplane Initialization and Selection

The selection block $\mathcal{S}_\omega$ first applies one cross-attention layer between the 768 learned triplane tokens T^0 and the retained prefix $Z_{:K}$. A LayerNorm–MLP uncertainty head assigns one score to each triplane token. Hard descending top-k selection retains the highest-scoring 25%, or 192 tokens, as H_K^0. The remaining 576 tokens form the bypassed state R_K and retain their spatial indices for subsequent triplane restoration. Selection is performed once before recurrent refinement rather than repeated at every pass.

The ranking indices are discrete, so gradients do not propagate through the top-k ordering itself. Reconstruction gradients propagate through the selected token features, while the uncertainty head receives direct supervision from $\mathcal{L}_{\text{unc}}$ defined below. The selected indices are retained to restore the refined tokens to their original triplane locations.

S3.3 Shared Refinement and Restoration

The shared refinement module is a six-layer Transformer block of width 512 with eight attention heads, GEGLU MLP ratio 2, and dropout 0.1. It has no causal mask, pass embedding, or pass-specific parameters. The recurrent sequence at pass ℓ is

$$U_K^{\ell} = [H_K^{\ell-1}; Z_{:K}] \in \mathbb{R}^{(192+K)\times 512}, \tag{S1}$$

where $H_K^{\ell-1}$ is the selected state and $Z_{:K}$ is the retained prefix. The shared block processes this sequence, but only its first 192 outputs define H_K^{ℓ}. The prefix is reinserted unchanged at every pass, while the bypassed tokens R_K remain fixed outside the shared block.

After the final pass, the updated selected tokens are scattered to their recorded spatial locations and combined with the 576 bypassed tokens at their original locations. A linear residual head maps each restored spatial token to an $8 \times 8 \times 32$ patch. The resulting residual is modulated by the predicted uncertainty and added to the corresponding initial triplane patch.

S3.4 Auxiliary Losses and Intermediate Supervision

The two inherited COD-VAE auxiliary losses stabilize triplane initialization and uncertainty-based selection. Let a_j^0 be the initial occupancy logit for query j, $\hat{y}_j^0 = \sigma(a_j^0)$ its probability, and $\hat{\mathcal{Y}}^0 = (\hat{y}_j^0)_{j=1}^{N_q}$ the corresponding probability vector. The initialization loss applies the reconstruction criterion to this initial prediction:

$$\mathcal{L}_{\text{init}} = \mathcal{L}_{\text{rec}}(\hat{\mathcal{Y}}^0, \mathcal{Y}). \tag{S2}$$

The uncertainty target is the detached pointwise binary cross-entropy of the initial prediction. Following COD-VAE, it is clipped and normalized to $[0, 1]$ before regression:

$$r_j = \frac{\text{clip}(\text{BCE}(\hat{y}_j^0, y_j) - 0.01, 0, 0.99)}{0.99}, \tag{S3}$$

$$\mathcal{L}_{\text{unc}} = \frac{1}{N_q} \sum_{j=1}^{N_q} (u_j - \text{sg}(r_j))^2, \tag{S4}$$

where u_j is the uncertainty field decoded at the query. We use $\lambda_{\text{init}} = 0.5$ and $\lambda_{\text{unc}} = 0.001$.

Stage-1 training unrolls six refinement passes. One intermediate depth is sampled uniformly from $\{1, \dots, 5\}$ for each minibatch, while prefix lengths are sampled independently for individual examples. The intermediate term uses $\beta = 0.5$, and its reconstruction and distillation components use the same volume and near-surface weights, 1.0 and 0.1, as the final reconstruction loss. Intermediate logits and detached final logits are clamped to $[-30, 30]$ before computing the distillation loss. The schedule α_t is updated once per optimizer step and remains fixed across the three microbatches accumulated into that update. Its endpoint T_α is the scheduled optimizer-step count after distributed sharding, dropping incomplete minibatches, and gradient accumulation. No additional triplane-feature distillation term is used.

S3.5 Stage-2 Prefix VAE

The prefix VAE accepts up to 16 ordered stage-1 tokens. A LayerNorm and linear projection independently map each 512-dimensional token to the mean and log variance of a 32-dimensional diagonal Gaussian posterior. The log variance is clamped to $[-30, 20]$, and training samples from the posterior by the reparameterization trick. A linear projection maps the sampled sequence back to width 512, after which learned positional embeddings and a 12-layer Transformer reconstruct the stage-1 token features. This Transformer uses eight heads, GEGLU MLP ratio 4, and dropout

0.1. Its causal mask is left-to-right:

$$A_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i, \end{cases} \tag{S5}$$

so the reconstruction at position i cannot use a later token.

Let $\widetilde{Z}$ denote a posterior sample and $\bar{Z} = D_\psi(\widetilde{Z})$ the reconstructed stage-1 token sequence. The stage-1 tokenizer is frozen while the prefix VAE is jointly optimized for $\mathcal{B} = \{1, 2, 4, 8, 16\}$ with weights $w_K \in \{1.2, 1.1, 1.0, 0.9, 0.8\}$. Its objective is

$$\begin{aligned} \mathcal{L}_{\text{pVAE}} = \sum_{K \in \mathcal{B}} & \frac{w_K}{\sum_{K' \in \mathcal{B}} w_{K'}} \\ & \times \Big[\text{MSE}\Big(\bar{Z}_{:K}, \text{LN}(Z_{:K})\Big) \\ & \qquad + \mathcal{L}_{\text{rec}}\Big(F_6(\bar{Z}_{:K}), \mathcal{Y}\Big)\Big] \\ & + 10^{-3} D_{\text{KL}}\Big(q_\psi(\widetilde{Z} \mid Z_{:16}) \,\|\, \mathcal{N}(0, I)\Big). \end{aligned} \tag{S6}$$

where F_6 denotes the frozen stage-1 decoder and occupancy head evaluated with six refinement passes. The feature and occupancy terms both have unit coefficients; the volume and near-surface weighting inside $\mathcal{L}_{\text{rec}}$ is given in Section S3.4. Causality ensures that every term for budget K depends only on the corresponding stage-2 prefix. The six-pass decoder is used only to supervise prefix VAE training. During generation evaluation, sampled codes are decoded using five refinement passes.

S3.6 Stage-2 Class-Conditional EDM

We cache deterministic posterior means for the full 16×32 latent array, normalize them using training-set channel statistics, and train the class-conditional EDM on arrays from all 55 ShapeNet training categories. Left-to-right causal self-attention ensures that the denoising prediction at each position depends only on the corresponding noisy prefix. At the reported two-token operating point, sampling therefore instantiates and denoises only the first two latent positions. The EDM and prefix VAE decoder thus operate on an exact 2×32 tensor without suffix padding. The EDM follows the latent-array design used by VecSet and COD-VAE [6, 7]: it has width 384, 16 Transformer blocks, eight 48-dimensional heads, GEGLU MLP ratio 4, no dropout, learned token-position embeddings, and a left-to-right causal self-attention mask. A 256-dimensional sinusoidal noise embedding is projected to width 384 and provides scale and shift to the adaptive LayerNorms in every block. The class index spans all 55 ShapeNet categories. It is mapped to a learned 384-dimensional embedding and supplied as a one-token key and value context through cross-attention in every block; token self-attention therefore models the latent sequence while class conditioning is injected separately.

For a normalized latent array x, category c, and $\log \sigma \sim \mathcal{N}(p_{\text{mean}}, p_{\text{std}}^2)$, the EDM objective is

$$\begin{aligned} \mathcal{L}_{\text{EDM}} = \mathbb{E}_{x,c,\sigma,\epsilon} & \Bigg[\frac{\sigma^2 + \sigma_{\text{data}}^2}{(\sigma \sigma_{\text{data}})^2} \\ & \quad \times \|D_\theta(x + \epsilon; \sigma, c) - x\|_2^2 \Bigg], \\ & \epsilon \sim \mathcal{N}(0, \sigma^2 I). \end{aligned} \tag{S7}$$

All latent positions present at an operating point are denoised in parallel; there is no token-autoregressive sampling loop.

S4 Evaluation Protocols

S4.1 Reconstruction Metric Settings

For both datasets, the encoder receives 2,048 points sampled from the stored surface pool of the target object. The task is therefore deterministic reconstruction from sampled target surfaces rather than completion from partial observations. For query IoU, the resulting representation is decoded at all 500,000 stored volume-query locations per object.

For mesh evaluation, the decoder is queried on a dense 128^3 occupancy grid. We extract the zero-logit level set, equivalently the 0.5-probability isosurface, with marching cubes. We then sample 100,000 points from the reconstructed mesh and compare them with the complete stored reference surface point cloud. Chamfer distance is the sum of the two mean bidirectional Euclidean nearest-neighbor distances. At threshold 0.02, precision and recall are the fractions of reconstructed and reference points, respectively, lying within the threshold of the other surface. F1 is their harmonic mean.

S4.2 Class-Conditioned Generation Metric Settings

Following COD-VAE [7], we evaluate airplane, car, chair, table, and rifle. We generate 2,000 shapes for each category and report minimum matching distance (MMD-CD), coverage (COV-CD), and 1-nearest-neighbor accuracy (1-NNA-CD). For a given category, the reference set S_r contains all objects of that category in the 1,283-object ShapeNet test split. From the 2,000 generated candidates, a deterministic subset of $5|S_r|$ shapes is used for MMD-CD and COV-CD, while a subset of $|S_r|$ shapes is used for 1-NNA-CD. The same reference sets and generated-set construction are used for every method.

Each mesh is represented by 2,048 points sampled from its surface. Using CD as the set distance, MMD averages the distance from each reference shape to its nearest generated shape. COV is the fraction of reference shapes selected as a nearest neighbor of at least one generated shape. The fixed 5:1 generated-to-reference ratio makes MMD-CD and COV-CD comparable across methods. 1-NNA is the leave-one-out nearest-neighbor classification accuracy on the union of equally sized reference and generated sets, avoiding sample-count imbalance between the two sets. We report the unweighted mean of each metric over the five categories.

VecSet and COD-VAE use 18 diffusion steps, whereas 3DILG uses its native 512-step autoregressive sampler. Our EDM uses 18 Heun steps with $\rho = 7$, $\sigma_{\max} = 80$, and $\sigma_{\min} = 0.002$, without condition dropout or classifier-free guidance.

S4.3 Efficiency Measurement Protocol

All efficiency measurements use the corresponding trained checkpoints on one NVIDIA H20 GPU with batch size 16 and FP32 inference. TF32 and Flash SDP are disabled. CUDA events measure GPU execution after warmup; model loading, data loading, CPU–GPU transfers, marching cubes, and file output are excluded. For reconstruction, full inference begins with the input point cloud and includes point encoding, tokenizer decoding, and all 128^3 occupancy queries in chunks of 131,072 points. Decoder latency instead begins with an already available stage-1 prefix and ends when the complete triplane has been produced; it excludes point encoding and occupancy queries.

For generation, sampling throughput measures EDM latent generation alone. Full throughput begins from the same latent noise and additionally includes prefix VAE decoding, five-pass stage-1 tokenizer decoding, and the complete 128^3 occupancy-field query. Thus the point encoder is part of reconstruction full inference but not of the generation pipeline. We discard three warmup batches and average ten measured batches for reconstruction and decoder latency; generation uses

Method	Tok.	Full shape/s	Peak GiB
VecSet	512	4.20	32.63
COD-VAE	2	82.48	2.08
COD-VAE	32	79.82	2.08
COD-VAE	64	77.29	2.09
ZipTok3D	1	72.51	2.03
ZipTok3D	2	72.51	2.03
ZipTok3D	4	72.24	2.03

Table B Full reconstruction efficiency under the common protocol. ZipTok3D uses five refinement passes.

five warmup and thirty measured batches. For a measured mean batch latency t in milliseconds,

$$\text{throughput} = \frac{16 \times 1000}{t}. \tag{S8}$$

Peak allocated CUDA memory is measured after resetting the peak-memory counter following warmup. It is measured over the corresponding full pipeline; reserved memory is not used.

S4.4 Reconstruction Efficiency Results

Table B reports the resulting representation–compute tradeoff. Shorter prefixes reduce neither the dense occupancy query nor the number of shared refinement calls. Consequently, ZipTok3D has lower end-to-end throughput than the single-pass COD-VAE variants, despite its shorter latent sequence. Peak memory remains similar because the common encoder and dense-field query dominate the allocation.

S4.5 Statistical Analysis

For the primary comparison between ZipTok3D at $K = 4, L = 5$ and COD-VAE-32, we resample aligned TRELLIS objects with replacement. Each of 20,000 resamples contains 2,613 object indices. For each metric, we compute the difference between the paired sample means and report the 2.5th and 97.5th percentiles of the resulting bootstrap distribution. The bootstrap uses seed 123456, and all effects are defined as ZipTok3D minus COD-VAE, so negative CD is favorable. These intervals quantify variation over evaluation objects conditional on the trained models; they do not measure variation across independent training runs.

S5 Post-Hoc Adaptive-Budget Diagnostic

To characterize how the required budget varies across individual shapes, we perform a post-hoc diagnostic over $K \in \{1, 2, 4, 8, 16, 32\}$ and $L \in \{1, \dots, 6\}$ from the same checkpoint. Candidates are searched by increasing K and then increasing L, prioritizing a shorter representation before shallower decoding. This diagnostic accesses ground-truth reconstruction metrics and thus measures adaptive potential rather than the performance of a deployable budget predictor.

We use a strict criterion that accepts a candidate only when all three metrics are no worse than COD-VAE-32 after decimal half-up rounding (one decimal for IoU/F1 and three decimals for CD). Shapes without a successful candidate are excluded from this diagnostic. Table C reports both methods on the same successful subset, preventing differences in subset composition from affecting the comparison.

No candidate in the evaluated grid satisfies the strict all-metric criterion for the remaining 304 ShapeNet and 846 TRELLIS objects. All fixed-budget evaluations include these objects; they are excluded only from the oracle averages in Table C. Consequently, the reported average budgets characterize the successful subset rather than the expected cost of a deployable early-exit policy.

Data	Method	Found	Rate	Avg. K	Avg. L
ShapeNet	COD-VAE-32	–	–	32.00	–
ShapeNet	ZipTok3D	979/1,283	76.31	2.43	3.41
TRELLIS	COD-VAE-32	–	–	32.00	–
TRELLIS	ZipTok3D	1,767/2,613	67.62	3.02	3.97

Data	Method	IoU↑	CD↓	F1↑
ShapeNet	COD-VAE-32	97.2	0.011	98.1
ShapeNet	ZipTok3D	97.3	0.011	98.3
TRELLIS	COD-VAE-32	75.8	0.015	96.1
TRELLIS	ZipTok3D	76.6	0.014	96.6

Table C All-metric post-hoc diagnostic. The ZipTok3D rows report the post-hoc oracle. Found and Rate give the number and percentage of shapes for which the search identifies an operating point that matches or exceeds COD-VAE-32 on all three rounded per-shape metrics. Avg. K and Avg. L are computed over these shapes. Metrics in the lower block are aggregated over the same subset for both methods, so the COD-VAE rows are subset rather than full-split results. IoU and F1 are percentages.

Figures D and E summarize the first successful budgets within this subset. Three or four passes account for 68.3% and 71.6% of the ShapeNet and TRELLIS subsets, respectively. This distribution is consistent with the dataset-level saturation behavior in Figures A and B.

S6 Additional Qualitative Comparisons

Figure F extends the qualitative comparison to objects with thin components, large openings, curved surfaces, and repeated architectural elements. The open shelf, chair, and table retain their principal empty regions, while the vessel and aircraft preserve separated components and overall topology. On TRELLIS, both reconstructions recover the multi-level building layouts and repeated roof or fortification structures. The remaining differences are concentrated around small protrusions, thin supports, roof eaves, and crenellations, which remain challenging under both token budgets.

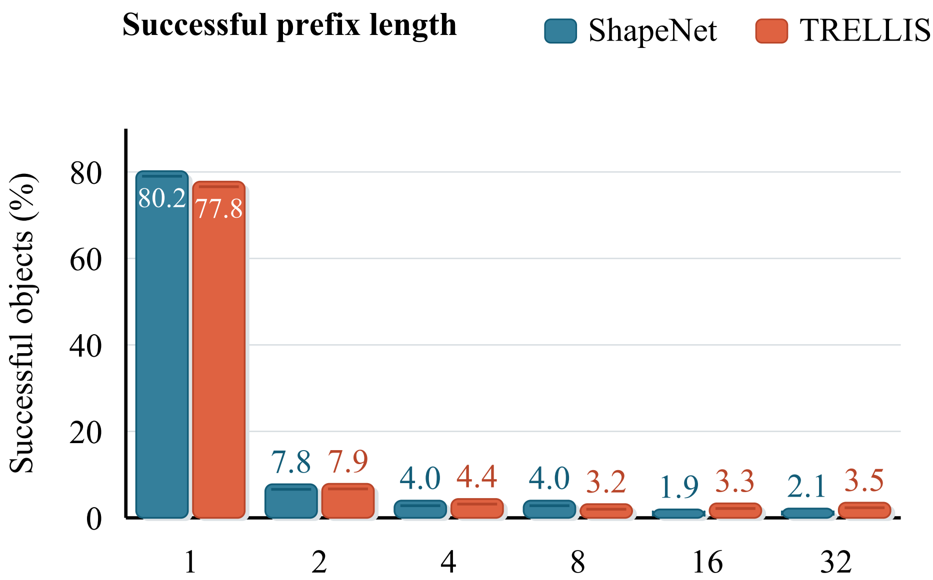

Figure D Distribution of the first successful prefix length K under the token-first all-metric search. Percentages are normalized within the successful subset of each dataset.

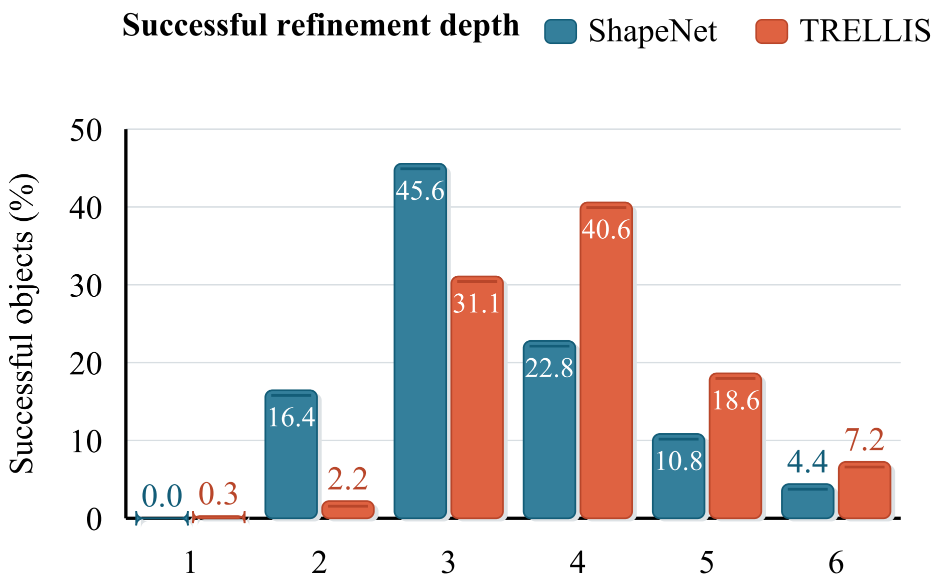

Figure E Distribution of the first successful refinement depth L under the same search and normalization.

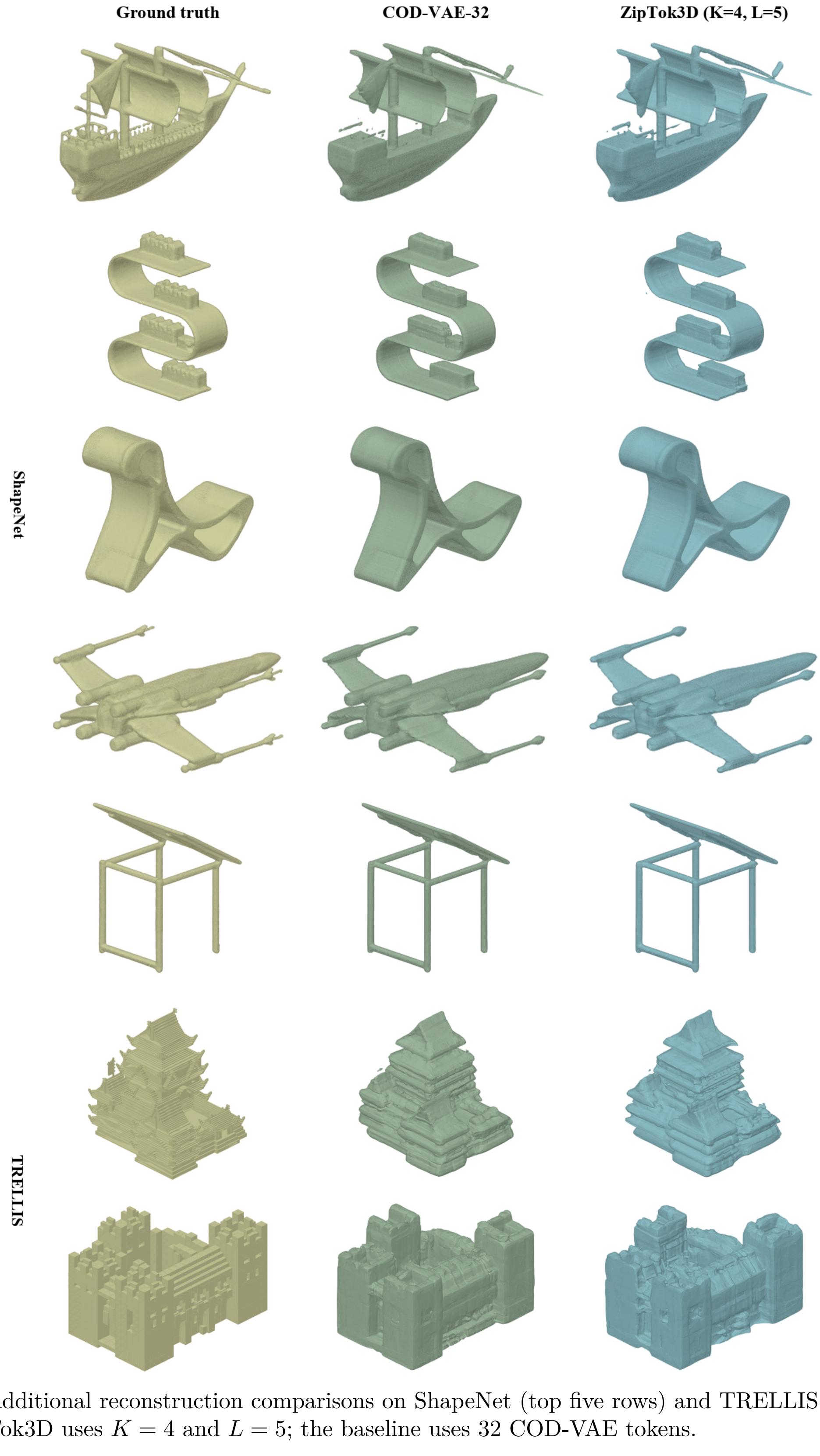

Figure F Additional reconstruction comparisons on ShapeNet (top five rows) and TRELLIS (bottom two rows). ZipTok3D uses $K = 4$ and $L = 5$; the baseline uses 32 COD-VAE tokens.

References

[1] Jinzhi Zhang, Feng Xiong, and Mu Xu. 3D representation in 512-byte: Variational tokenizer is the key for autoregressive 3D generation. *arXiv preprint arXiv:2412.02202*, 2024.

[2] Kangle Deng, Hsueh-Ti Derek Liu, Yiheng Zhu, Xiaoxia Sun, Chong Shang, Kiran S Bhat, Deva Ramanan, Jun-Yan Zhu, Maneesh Agrawala, and Tinghui Zhou. Efficient autoregressive shape generation via octree-based adaptive tokenization. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11685–11696, 2025.

[3] Niladri Shekhar Dutt, Zifan Shi, Paul Guerrero, Chun-Hao Paul Huang, Duygu Ceylan, Niloy J Mitra, and Xuelin Chen. LoST: Level of semantics tokenization for 3D shapes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19950–19959, 2026.

[4] Yuan Li, Congyi Zhang, Xifeng Gao, and Xiaohu Guo. SuperVoxelGPT: Adaptive and ordered 3D tokenization for autoregressive shape generation. *arXiv preprint arXiv:2605.29655*, 2026.

[5] Biao Zhang, Matthias Nießner, and Peter Wonka. 3DILG: Irregular latent grids for 3D generative modeling. *Advances in Neural Information Processing Systems*, 35:21871–21885, 2022.

[6] Biao Zhang, Jiapeng Tang, Matthias Nießner, and Peter Wonka. 3DShape2VecSet: A 3D shape representation for neural fields and generative diffusion models. *ACM Transactions on Graphics (TOG)*, 42(4):1–16, 2023.

[7] In Cho, Youngbeom Yoo, Subin Jeon, and Seon Joo Kim. Representing 3D shapes with 64 latent vectors for 3D diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 28556–28566, 2025.

[8] Biao Zhang and Peter Wonka. LaGeM: A large geometry model for 3D representation learning and diffusion. In *International Conference on Learning Representations*, 2025.

[9] Oren Rippel, Michael Gelbart, and Ryan Adams. Learning ordered representations with nested dropout. In *International Conference on Machine Learning*, pages 1746–1754. PMLR, 2014.

[10] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. *arXiv preprint arXiv:1807.03819*, 2018.

[11] Sahil Goyal, Swayam Agrawal, Gautham Govind Anil, Prateek Jain, Sujoy Paul, and Aditya Kusupati. ELT: Elastic looped transformers for visual generation. *arXiv preprint arXiv:2604.09168*, 2026.

[12] Roman Bachmann, Jesse Allardice, David Mizrahi, Enrico Fini, Oğuzhan Fatih Kar, Elmira Amirloo, Alaaeldin El-Nouby, Amir Zamir, and Afshin Dehghan. FlexTok: Resampling images into 1D token sequences of flexible length. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 2241–2292. PMLR, 2025.

[13] Xiaohui Zeng, Arash Vahdat, Francis Williams, Zan Gojcic, Or Litany, Sanja Fidler, and Karsten Kreis. LION: Latent point diffusion models for 3D shape generation. *Advances in Neural Information Processing Systems*, 35:10021–10039, 2022.

[14] Bojun Xiong, Si-Tong Wei, Xin-Yang Zheng, Yan-Pei Cao, Zhouhui Lian, and Peng-Shuai Wang. OctFusion: Octree-based diffusion models for 3D shape generation. In *Computer Graphics Forum*, volume 44, page e70198. Wiley Online Library, 2025.

[15] Xuanchi Ren, Jiahui Huang, Xiaohui Zeng, Ken Museth, Sanja Fidler, and Francis Williams. XCube: Large-scale 3D generative modeling using sparse voxel hierarchies. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4209–4219, 2024.

[16] Jianfeng Xiang, Zelong Lv, Sicheng Xu, Yu Deng, Ruicheng Wang, Bowen Zhang, Dong Chen, Xin Tong, and Jiaolong Yang. Structured 3D latents for scalable and versatile 3D generation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 21469–21480, 2025.

[17] Zeqiang Lai, Yunfei Zhao, Zibo Zhao, Haolin Liu, Qingxiang Lin, Jingwei Huang, Chunchao Guo, and Xiangyu Yue. Lattice: Democratize high-fidelity 3d generation at scale. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19982–19992, June 2026.

[18] Jianfeng Xiang, Xiaoxue Chen, Sicheng Xu, Ruicheng Wang, Zelong Lv, Yu Deng, Hongyuan Zhu, Yue Dong, Hao Zhao, Nicholas Jing Yuan, and Jiaolong Yang. Native and compact structured latents for 3d generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14419–14429, June 2026.

[19] Mohammad Asim, Christopher Wewer, and Jan Eric Lenssen. Scenetok: A compressed, diffusable token space for 3d scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5870–5880, 2026.

[20] Rui Chen, Jianfeng Zhang, Yixun Liang, Guan Luo, Weiyu Li, Jiarui Liu, Xiu Li, Xiaoxiao Long, Jiashi Feng, and Ping Tan. Dora: Sampling and benchmarking for 3D shape variational auto-encoders. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16251–16261, 2025.

[21] Jen-Hao Rick Chang, Yuyang Wang, Miguel Angel Bautista Martin, Jiatao Gu, Xiaoming Zhao, Josh Susskind, and Oncel Tuzel. 3d shape tokenization via latent flow matching. *arXiv preprint arXiv:2412.15618*, 2024.

[22] Aadarsh Sahoo, Vansh Tibrewal, and Georgia Gkioxari. Aligning text, images and 3d structure token-by-token. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14905–14914, June 2026.

[23] Zeqiang Lai, Yunfei Zhao, Zibo Zhao, Haolin Liu, Fuyun Wang, Huiwen Shi, Xianghui Yang, Qingxiang Lin, Jingwei Huang, Yuhong Liu, Jie Jiang, Chunchao Guo, and Xiangyu Yue. Unleashing vecset diffusion model for fast shape generation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 2523–2533, October 2025.

[24] Xin Wen, Bingchen Zhao, Ismail Elezi, Jiankang Deng, and Xiaojuan Qi. ”principal components” enable a new language of images. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 16641–16651, October 2025.

[25] Yitong Chen, Zuxuan Wu, Xipeng Qiu, and Yu-Gang Jiang. Catok: Taming mean flows for one-dimensional causal image tokenization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 23161–23171, June 2026.

[26] Shivam Duggal, Phillip Isola, Antonio Torralba, and William T. Freeman. Adaptive length image tokenization via recurrent allocation. In *International Conference on Learning Representations*, 2025.

[27] Wilson Yan, Volodymyr Mnih, Aleksandra Faust, Matei Zaharia, Pieter Abbeel, and Hao Liu. ElasticTok: Adaptive tokenization for image and video. In *International Conference on Learning Representations*, 2025.

[28] Yan Li, Changyao Tian, Renqiu Xia, Ning Liao, Weiwei Guo, Hongsheng Li, Jifeng Dai, Hao Li, and Xue Yang. AdapTok: Learning adaptive and temporally causal video tokenization in a 1D latent space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16163–16172, 2026.

[29] Tianwei Xiong, Jun Hao Liew, Zilong Huang, Zhijie Lin, Jiashi Feng, and Xihui Liu. Evatok: Adaptive length video tokenization for efficient visual autoregressive generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 23249–23259, June 2026.

[30] Andrei Atanov, Jesse Allardice, Roman Bachmann, Oğuzhan Fatih Kar, R Devon Hjelm, David Griffiths, Peter Fu, Afshin Dehghan, and Amir Zamir. Videoflextok: Flexible-length coarse-to-fine video tokenization. In *Forty-third International Conference on Machine Learning*, 2026.

[31] Zixuan Fu, Lanqing Guo, Chong Wang, Binbin Song, Ding Liu, and Bihan Wen. Improving flexible image tokenizers for autoregressive image generation. *arXiv preprint arXiv:2601.01535*, 2026.

[32] Xiaogang Wang, Marcelo H Ang Jr, and Gim Hee Lee. Cascaded refinement network for point cloud completion. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 790–799, 2020.

[33] Tianxin Huang, Hao Zou, Jinhao Cui, Xuemeng Yang, Mengmeng Wang, Xiangrui Zhao, Jiangning Zhang, Yi Yuan, Yifan Xu, and Yong Liu. RFNet: Recurrent forward network for dense point cloud completion. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 12508–12517, 2021.

[34] Angel X Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, et al. ShapeNet: An information-rich 3D model repository. *arXiv preprint arXiv:1512.03012*, 2015.